Palimpsest NYC: A Citation-Grounded Agentic Walking-Tour System for Manhattan

Chenhao Yang
Columbia University
New York, NY, USA
cy2822@columbia.edu

Kaining Jia
Columbia University
New York, NY, USA
kj2712@columbia.edu

Thomas Duan
Columbia University
New York, NY, USA
hd2593@columbia.edu

ABSTRACT

Palimpsest NYC is an agentic walking-tour system that answers place-based questions about a bounded region of New York City and turns them into citation-grounded narrations and optional walking routes. The system combines a FastAPI backend, a retrieval-augmented agent loop, PostgreSQL with PostGIS and pgvector, OpenStreetMap and Wikipedia-derived content, and a React plus MapLibre frontend that streams results over Server-Sent Events. The goal is not just fluent text; every user-facing claim must be backed by a retrieved document. We enforce this with a narrow two-tool surface, a strict terminal JSON schema, and a citation verifier that checks every cited document against a per-conversation retrieval ledger. The current version adds richer map interactions, true multi-turn session memory, and a food discovery workflow that lets users compare nearby cafes or restaurants on the map and then continue the route to a chosen destination. The result is a coherent end-to-end application rather than a collection of disconnected components: retrieval, reasoning, citation, routing, streaming, and visualization live inside one user experience. We pre-register a 95-question `manhattan-100` bank and ablate five systems on it. The hybrid configuration is the strongest Palimpsest variant for citation grounding; the cross-encoder reranker trades a within-CI CCR gain for a +17.4 pp hallucination-rate regression and a 0.500 to 0.200 drop in out-of-scope refusal.

1 INTRODUCTION AND PROBLEM FORMULATION

Large language models are now strong enough to produce convincing urban narratives, but a city-guide application has a higher bar than general chat. It has to connect generated text to real places, stay geographically bounded, and expose enough structure that a user can verify where claims come from. In a walking-tour setting, a hallucinated location or an unsupported claim directly weakens trust. Palimpsest NYC works inside a narrow but realistic domain: short walking tours in Manhattan, grounded in public-domain sources.

The core question we ask is: *can we build an end-to-end LLM application that produces useful, map-aware urban narration while enforcing citation correctness at generation time rather than treating grounding as a post-hoc add-on?* Our answer is a constrained agentic system in which retrieval, routing, and rendering are all first-class components. The backend retrieves candidate places and documents, the agent chooses whether to search or plan a route, a verifier checks the final citation set, and the frontend renders the narration and the route as one synchronized experience.

This version extends the base MVP in four ways. First, the map is no longer a passive visualization; citations, route stops, and markers highlight one another bidirectionally. Second, the interface supports true multi-turn sessions, so follow-up instructions such as "make it shorter" or "focus on churches" operate on preserved history instead of restarting a new request from scratch. Third, the system supports a structured food discovery flow that lets a user ask for coffee, lunch, or dessert nearby, inspect candidate places on the map, and continue the walk to a chosen stop. Fourth, we ship a pre-registered `manhattan-100` evaluation: a 95-question Manhattan bank ablated across five systems (`vanilla`, `naive_rag`, and three Palimpsest configurations that differ only in retrieval mode), with 95% bootstrap CIs over CCR, HR, NQ, FA, and GRR, which puts numbers on the retrieval-mode choice rather than leaving it to intuition.

2 RELATED WORK AND BACKGROUND

Palimpsest NYC draws on three lines of work: retrieval-augmented generation, geographic data ingestion, and street-level route planning. The retrieval stack uses BGE sentence embeddings [10] stored in `pgvector` [7], fused with sparse `pg_trgm` name similarity [13] through Reciprocal Rank Fusion [11], with an optional BGE cross-encoder reranker [14] over the top-k union. We treat citation grounding as a contract enforced at generation time against a per-conversation retrieval ledger rather than as a post-hoc decoration, which lets us measure the contract empirically (Section 5).

Geographic content comes from OpenStreetMap via the Overpass API [4, 15] for named places and amenities, and from Wikipedia [5] for longer-form historical context. Routing is delegated to OSRM [3] so the LLM never improvises paths; the agent chooses which place IDs matter, and OSRM returns street-following geometry and turn-by-turn steps. The bounded geography keeps retrieval tractable and makes a citation-grounded narrative well-defined over a finite corpus.

3 SYSTEM DESIGN AND METHODOLOGY

Palimpsest NYC follows a monorepo architecture with three application surfaces: a FastAPI backend, a minimal worker process, and a React frontend. The backend exposes the key interfaces, including a health endpoint, a general chat endpoint, and the streamed agent endpoint. The frontend is responsible for user interaction, map rendering, and progressive display of server events. The worker is intentionally lightweight in this version and leaves ingestion as an explicit command-line task.

At the center of the system is an agent loop with a deliberately narrow contract. The loop exposes only two callable tools: `search_places` for retrieval and `plan_walk` for route construction. The agent operates under a hard turn cap and must terminate by emitting JSON with two top-level fields: `narration` and `citations`.

Table 1: Core system components and responsibilities.

Component	Responsibility
React + MapLibre frontend	Collect user queries, persist session state, stream SSE results, render citations, routes, and map interactions.
FastAPI agent backend	Run the LLM loop, dispatch tools, verify citations, frame SSE events, and expose the application API.
PostgreSQL + PostGIS + pgvector	Store places and documents, support semantic retrieval, spatial lookups, and provenance-preserving records.
`search_places` tool	Retrieve relevant place and document candidates from the bounded corpus.
`plan_walk` tool	Convert cited place IDs into street-following routes and step-by-step walking instructions via OSRM.
Citation verifier	Enforce the five-field citation contract against the retrieval ledger before a response reaches the user.
Retrieval factory	Boot-time selection of dense / hybrid / hybrid-reranked retrieval modes via the `RETRIEVAL_MODE` env flag; preserves the `SearchPlaceHit` tool-result shape so the agent loop is mode-agnostic.

Each citation must contain exactly five required fields: `doc_id`, `source_url`, `source_type`, `span`, and `retrieval_turn`. A verifier then checks this citation list against the per-session retrieval ledger. If verification fails, the loop allows one corrective retry; if the second attempt still fails, the system surfaces a warning instead of silently pretending success.

This design constrains the model enough to make failure modes legible. The LLM can reason about content and route intent, but it cannot invent an arbitrary tool, skip the output schema, or cite an unseen record without being caught by the verifier. On the client side, the backend streams typed events over SSE, including tool calls, tool results, narration, citations, and route information. This makes the system easier to observe and easier to debug than a single opaque response.

A `RetrievalMode` factory reads the `RETRIEVAL_MODE` env flag at boot and selects one of three implementations. The *dense* mode runs a pgvector cosine search over 384-dim `BAAI/bge-small-en-v1.5` embeddings indexed with ivfflat (`lists=100`) [10]. The *hybrid* mode adds a sparse branch over `pg_trgm` name similarity [13] and fuses the two rankings with Reciprocal Rank Fusion at $k = 60$ [11]. The *hybrid-reranked* mode keeps that fusion step and then re-scores the top-k union with a BGE-reranker-base cross-encoder [14]. All three modes return the same `SearchPlaceHit` tool-result shape, so the agent loop never sees the mode: swapping it changes ranking quality, not the tool contract or the verifier path.

Several later features build naturally on top of this architecture. The multi-turn session design serializes user and assistant history and sends it with each new request, allowing the loop to revise, shorten, or redirect prior answers. The map interaction layer maintains shared focus state between markers, citation cards, and the route timeline, so the same stop can be located from multiple interface surfaces. The food discovery feature adds a structured discovery endpoint that returns candidate dining places and then turns user selection into a follow-up routing request inside the existing session model.

4 IMPLEMENTATION AND DATA

The backend is implemented in Python 3.12 with FastAPI and async SQLAlchemy. The database layer uses PostgreSQL 16 with PostGIS for geometry, pgvector for embedding search, and trigram support for lexical matching. The ingestion pipeline populates the local corpus from OpenStreetMap (via the Overpass API) for named places, amenities, and coordinates, and from Wikipedia / Wikidata for longer-form historical context; the food discovery extension widens the OSM tag set to cover restaurants, cafes, bakeries, and similar venues.

The corpus has been widened from the original Morningside Heights / Upper West Side slice to all of Manhattan. Ingestion runs against the bounding box $(-74.02, 40.70, -73.91, 40.88)$ under `SCOPE_VERSION = v2-manhattan`, and after de-duplication yields **12,858** OSM places, **492** Wikipedia places, and **456** Wikipedia documents. On the OSM side that is a **9.4×** expansion over the V1 MH/UWS slice (1,363 places). The Wikipedia side is flat at 492 because the Wikipedia ingest is API-capped at 500 items per batch; the cap binds before the geographic filter, so the 492 final count matching the V1 figure is a cap artifact rather than a coincidence of geography. Lifting the cap is a v3 follow-up. Table 2 reports the counts.

Table 2: Manhattan corpus counts after de-duplication at `SCOPE_VERSION = v2-manhattan`.

Source	Count
OSM places	12,858
Wikipedia places	492
Wikipedia documents	456

Embeddings are produced with a 384-dimensional sentence-transformer model. That choice keeps inference light enough for the project scale while still supporting semantic retrieval over the place and document tables. Importantly, the database schema is migration-first: SQL migrations define the authoritative schema, and ORM models mirror that schema rather than creating it dynamically at runtime. This keeps the data layer predictable and reproducible.

The frontend is implemented with React, Vite, and TypeScript. Instead of treating the map as a static image pane, the interface integrates narration, citations, route stops, and place candidates into a shared interaction model. When a citation card is selected, the map can fly to the corresponding stop; when a marker is selected, the citation list and route timeline can reveal the same place. For food discovery, candidate markers appear before a route has been finalized, allowing the user to inspect alternatives visually before choosing a destination.

The LLM call path has two router tiers, `local` and `openrouter`, each behind its own circuit breaker configured for **3 failures** inside a **60-second window** with a **30-second cooldown**. Bring-your-own-key (BYOK) requests carry credentials in an `X-LLM-Credentials` header; the backend reads that header per request and constructs a *per-session* router with fresh breaker state and an isolated cache namespace, so a user-supplied bad key cannot trip the shared breaker or poison the shared LLM cache for other sessions. The agent loop also gives `plan_walk` an along-route discovery step: alongside the OSRM walking geometry between cited places, it surfaces nearby POIs as candidate stops on that route. This folds route-aware discovery into the existing two-tool surface rather than adding a third tool, and the discovered stops flow through the same citation ledger as any other retrieved place.

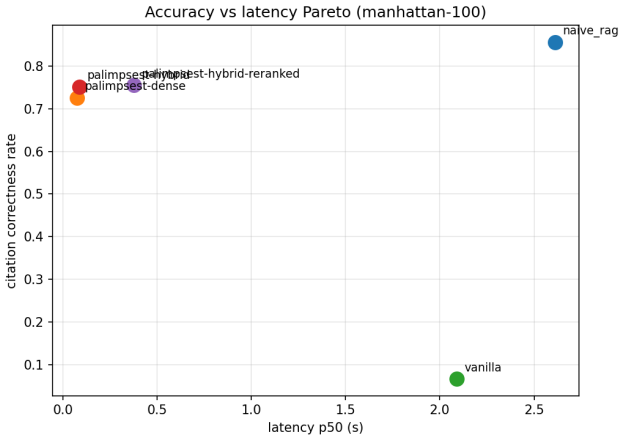

Figure 1: Pareto frontier of CCR vs. p50 latency across the five systems. All systems share a Redis-backed prompt cache (`LLMCache`); plotted p50 latencies reflect warm-cache timing. Cold Palimpsest is approximately 2–4 s per question over 2–4 tool turns. Because all three Palimpsest configurations serve the same cached prompts, the relative ordering between them remains meaningful.

5 EVALUATION AND RESULTS

We evaluate Palimpsest against the pre-registered `manhattan-100` question bank (tagged `eval/manhattan-100-v1`). The bank contains 95 questions: 85 in-scope items spanning roughly twenty Manhattan neighborhood tags plus the `varied` and `unknown` buckets, and 10 out-of-scope controls (Brooklyn, Queens, Bronx, out-of-state, and fictional places) used to score refusal behavior. We froze the bank before running any of the five systems against it.

The ablation compares five systems: `vanilla` (LLM with no retrieval), `naive_rag` (top-k retrieval concatenated into the prompt), `palimpsest-dense`, `palimpsest-hybrid`, and `palimpsest-hybrid-reranked`. All five are served by `openai/gpt-5.4-mini` through OpenRouter, so the model is held constant; the three Palimpsest variants differ only by the `RETRIEVAL_MODE` boot flag and the reranker-enabled toggle.

We report five metrics with 95% bootstrap confidence intervals over 1000 resamples: CCR (citation-correctness rate), HR (hallucination rate, lower is better), FA (false-attribution count per response), NQ (narration quality on a 1–5 Likert), and GRR (grounded refusal rate on the out-of-scope subset, higher is better). The judge is `gpt-5.4-mini` served through OpenRouter; this is the same model class as the systems under evaluation, which is a self-grading risk we flag in the methodology caveats below. Total judge spend is approximately $1.60 over roughly 1550 calls.

Naive RAG posts the highest CCR in the table at 0.856, but this is a construction artifact rather than a quality claim. `naive_rag` concatenates the entire top-k retrieved set verbatim into the prompt, so any `doc_id` the model cites is present in the retrieved set by construction; CCR collapses to "did the model cite something from the block of text we just handed it." The three Palimpsest configurations instead invoke retrieval as agent tools across multiple turns and cite a smaller, targeted subset per turn, so their CCR is a stricter test of citation grounding. The meaningful comparison is therefore among the Palimpsest variants.

Moving from `palimpsest-dense` to `palimpsest-hybrid` yields +2.6 pp CCR (0.725 → 0.751) and −8.8 pp HR (0.708 → 0.620). RRF fusion ($k = 60$) of dense `bge-small` cosine scores with sparse `pg_trgm` name similarity recovers proper-name lookups that pure dense embeddings miss. We tracked two diagnostic cases through the run: Inwood Hill Park and the Lower East Side Tenement Museum. Both are present in the corpus, but the dense embedding ranks similarly-named community gardens and other `tenement`-prefixed entries above them, and hybrid retrieval pulls the canonical record back to the top of the result set. FA also moves with this transition (hybrid FA = 1.422 vs dense FA = 1.029); the agent surfaces more candidates and occasionally over-attributes.

The `palimpsest-hybrid` → `palimpsest-hybrid-reranked` move is the most unflattering line in the table, and we report it plainly. CCR rises by +0.4 pp (0.751 → 0.755), well inside the bootstrap CI. HR regresses by +17.4 pp (0.620 → 0.794), and GRR collapses from 0.500 to 0.200 on the 10 out-of-scope controls. FA does soften slightly (reranker FA = 1.176 vs hybrid FA = 1.422), so the reranker reduces over-attribution somewhat on the cases where it cites at all; vanilla FA = 0.989 is misleading because vanilla almost never cites. Our reading is that reranker tuning is an *agent-prompt* concern, not just a retrieval-quality concern: when the cross-encoder surfaces more topically-relevant candidates, the agent treats them as license to make more claims, including on questions where it should refuse. The fix belongs in the agent prompt that adjudicates which retrieved candidates make it into the citation set, not in the retriever stack.

Five methodology caveats bound how strongly these numbers should be read:

- **v2 CCR rubric.** OSM rows store name, URL, and coordinates but no prose, so `body_excerpt` is empty by corpus design. The v2 rubric (`prompt_version: v2`) accepts `doc_id` ∈ `retrieved_docs` as sufficient grounding when the body is empty; v1 of the rubric scored every OSM citation to 0 and is no longer trustworthy for this corpus.

Table 3: Headline ablation on the pre-registered `manhattan-100` bank ($n = 95$); mean [95% bootstrap CI]. FA omitted (see prose).

System	n	CCR	HR ↓	NQ	GRR ↑
`vanilla`	95	0.068 [0.024, 0.126]	0.186 [0.136, 0.236]	3.99 [3.89, 4.09]	0.100 [0.000, 0.300]
`naive_rag`	95	0.856 [0.791, 0.914]	0.313 [0.242, 0.388]	3.41 [3.24, 3.59]	0.700 [0.400, 1.000]
`palimpsest-dense`	95	0.725 [0.648, 0.797]	0.708 [0.621, 0.786]	3.44 [3.29, 3.58]	0.400 [0.100, 0.700]
`palimpsest-hybrid`	95	0.751 [0.683, 0.815]	0.620 [0.530, 0.717]	3.45 [3.30, 3.59]	0.500 [0.200, 0.800]
`palimpsest-hybrid-reranked`	95	0.755 [0.678, 0.818]	0.794 [0.722, 0.860]	3.52 [3.39, 3.64]	0.200 [0.000, 0.500]

- **Doc-ID harvesting.** We reconstruct `retrieved_docs` from the terminal `citations` frame and from `walk.discovered_stops[]`, not from a tool-result ledger: the V1 SSE schema only exposes `{name, n_hits}` per tool call. This undercounts the HR denominator, because any retrieved-but-uncited document is invisible to the eval.
- **LLMCache contamination.** All five systems share a Redis-backed prompt cache across the 95-question run. Correctness metrics (CCR/HR/FA/NQ/GRR) are unaffected, but latency and cost are deflated asymmetrically across systems and turn counts; the Pareto figure (Figure 1) should be read as warm-cache shape, not cold-start cost.
- **Self-grading judge.** `gpt-5.4-mini` is both the system model and the judge model. This is a documented self-grading risk and is not mitigated by an independent grader in this run; an independent judge (e.g., `claude-opus-4-7`) would resolve it at ∼3–5× judge cost via a one-flag change to `judge_run.py`.
- **Inter-rater κ deferred.** The `kappa` cell in the ablation table is `null`. Hand-grading the calibration worksheet at `docs/eval/grades/calibration.csv` and re-running `aggregate.py --hand-grades` recovers the number; we defer that step in this revision.

6 DISCUSSION, LIMITATIONS, AND INSIGHTS

The headline finding from the ablation is that turning on the cross-encoder reranker on top of hybrid retrieval moves CCR by only +0.4 pp (within CI) while regressing HR by +17.4 pp and dropping out-of-scope refusal (GRR) from 0.500 to 0.200. CCR stays roughly flat because the reranker reshuffles top-k order without changing which documents are available. HR rises because the agent reads the reshuffled ordering as license to make more, longer factual claims that it then cannot ground in the OSM corpus. The takeaway is that reranker tuning in an agent-grounded RAG pipeline is an *agent-prompt* concern, not just a retrieval-quality concern: the same retrieval set can produce very different grounding behavior depending on what the agent is encouraged to do with it. Plain hybrid remains the strongest Palimpsest configuration for citation grounding in this run.

Four evaluation caveats matter. The judge is `gpt-5.4-mini`, the same model that drives the systems under test, which is a known self-grading risk we accepted to keep the judge bill near $1.60 across 1550 calls; inter-rater κ against hand grades is deferred (the `kappa` cell is `null` in the aggregate table, recoverable by re-running `aggregate.py --hand-grades`). The Redis-backed LLMCache contaminates wall-clock latency and cost across re-runs of the same question but does not affect the correctness metrics (CCR, HR, FA, NQ, GRR), which evaluate the same answer either way. FA is CI-noisy at $n = 95$, so the reranker's FA = 1.18 against hybrid's FA = 1.42 should be read as suggestive rather than decisive.[1]

Product limitations from earlier versions still apply. The geographic scope is bounded by design to a Manhattan bounding box, and recommendation quality for food discovery is tied to OpenStreetMap tag freshness and does not include live business metadata such as ratings, hours, or occupancy. The strongest reliability gains continue to come from the surrounding system design (narrow tool surface, locked citation schema, deterministic OSRM routing) rather than from the model alone.

7 CONCLUSION

Palimpsest NYC shows that an LLM-based city-guide application can be both interactive and structured. The project combines retrieval, routing, citation verification, and map-based visualization into one end-to-end workflow for a bounded urban region. The technical contribution is less about the use of an LLM and more about the way the surrounding system constrains and validates it: a narrow two-tool surface, a locked citation schema verified against a per-conversation ledger, and a configurable retrieval factory (dense, hybrid, hybrid-reranked) that lets us compare grounding strategies on equal footing. The final result is a working agentic application that produces citation-grounded narration, route-aware map experiences, multi-turn refinements, and structured place discovery while preserving a coherent architecture across backend, database, and frontend layers, validated by a pre-registered ablation that puts numbers on the grounding-vs-breadth trade-off.

PROJECT LINKS

GitHub: https://github.com/nyavana/Palimpsest-NYC
Project website: https://palimpsest-nyc.nyavana.io/
Online demo: https://palimpsest-demo.nyavana.io/
Demo video: https://www.youtube.com/watch?v=RdxTcexnCRo

REFERENCES

[1] Sebastian Ramirez. FastAPI. https://fastapi.tiangolo.com/
[2] MapLibre Contributors. MapLibre GL JS. https://maplibre.org/
[3] Project OSRM. Open Source Routing Machine. https://project-osrm.org/
[4] OpenStreetMap Contributors. OpenStreetMap. https://www.openstreetmap.org/
[5] Wikimedia Foundation. Wikipedia. https://www.wikipedia.org/
[6] The PostgreSQL Global Development Group. PostgreSQL. https://www.postgresql.org/
[7] pgvector Contributors. pgvector. https://github.com/pgvector/pgvector
[8] OpenRouter. OpenRouter API. https://openrouter.ai/

[1] See `docs/eval/v1-eval-report.md` for the V1 Kimi K2.6 vs. GPT-OSS-120B router-bench (cost/latency, different question set; not reported here).

[9] Association for Computing Machinery. ACM article template. https://www.overleaf.com/latex/templates/association-for-computing-machinery-acm-sigplan-proceedings-template/rfvsrhgmghtc

[10] Shitao Xiao, Zheng Liu, Peitian Zhang, and Niklas Muennighoff. C-Pack: Packaged Resources To Advance General Chinese Embedding. arXiv:2309.07597, 2023. https://arxiv.org/abs/2309.07597

[11] Gordon V. Cormack, Charles L. A. Clarke, and Stefan Büttcher. Reciprocal Rank Fusion outperforms Condorcet and individual rank learning methods. In *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR '09)*, pages 758–759, 2009. https://doi.org/10.1145/1571941.1572114

[12] Stephen Robertson and Hugo Zaragoza. The Probabilistic Relevance Framework: BM25 and Beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389, 2009. https://doi.org/10.1561/1500000019

[13] The PostgreSQL Global Development Group. pg_trgm — support for similarity of text using trigram matching. PostgreSQL documentation. https://www.postgresql.org/docs/current/pgtrgm.html

[14] Shitao Xiao, Zheng Liu, Peitian Zhang, and Niklas Muennighoff. BGE-reranker-base: a cross-encoder reranker from the FlagEmbedding project. BAAI / Hugging Face model card. https://huggingface.co/BAAI/bge-reranker-base

[15] OpenStreetMap Foundation. Overpass API. https://overpass-api.de/